



In Fulfillment of the Requirements of COEN 330

Machine Learning

Section AA

Project Title:

Loan Approval Prediction Model

Group ID:

Team 1

By:

Jacques Ivan Cacatian-Martinez, 40291760

Djessica Ndjiongue Djapoum, 40263245

Mekoupou Eko Candys Eve, 40265277

Aaida Hossain, 40281796

Dipita Sinha, 40273009

Due: June 27th, 2026

"We certify that this submission is our original work and meets the Faculty's Expectations of Originality."

06/27/2026

Table of Content

Abstract.....	2
Introduction and Motivation	3
Dataset Description.....	6
Preprocessing and Exploratory Data Analysis.....	7
Methodology and Models	15
Validation and Hyperparameter Tuning Strategy	17
Figure 15. Threshold tuning for Gaussian Naive Bayes (out-of-fold).	20
Experimental Setup.....	21
Results and Model Comparison.....	22
Error analysis and Qualitative Discussion	26
1. Error Breakdown	26
2. Patterns in Misclassification.....	28
Demo or Usage Demonstration Description	30
Limitations and Future Work.....	34
Conclusion	36
Team Contribution	37
<i>Jacques Ivan Cacatian-Martinez</i>	37
<i>Djessica Ndjiongue Djapoum</i>	37
<i>Mekoupou Eko Candys Eve</i>	37
<i>Aaida Hossain</i>	37
<i>Dipita Sinha</i>	37
References.....	38
GitHub	40

Abstract

Loan approval is a critical decision-making process for financial institutions, requiring a balance between minimizing financial risk and approving qualified applicants. In this project, loan approval prediction is formulated as a supervised binary classification problem using a publicly available dataset containing 45,000 loan applications described by demographic, financial, and credit-related features. Because the dataset is imbalanced, with approximately 78% rejected applications and 22% approved applications, the study focuses on accurately identifying the minority approved class rather than maximizing overall accuracy.

A complete machine learning pipeline was developed, including data preprocessing, exploratory data analysis, feature transformation, categorical encoding, numerical scaling, and leakage-safe model training. Five classification models: Logistic Regression, Gaussian Naive Bayes, Support Vector Machine (SVM), Random Forest, and Gradient Boosting (histogram-based), along with a Dummy Classifier were implemented and compared. Hyperparameter tuning was performed using 5-fold stratified cross-validation with grid search, where the F1-score of the approved class was used both for model selection and to set the final decision threshold. Exploratory analysis revealed that the **previous_loan_defaults_on_file** feature was an exceptionally strong predictor of the target variable; consequently, all models were evaluated both with and without this feature to assess its influence on predictive performance.

The comparative evaluation shows that gradient boosting performs best ($F1 = 0.846$ on the approved class), and that careful preprocessing, appropriate validation, and ensemble learning methods provide robust performance for loan approval prediction while handling class imbalance through class weights and threshold tuning. The study also highlights the importance of analysing feature dominance and model behaviour rather than relying solely on overall accuracy, providing a more reliable framework for developing machine learning decision-support systems in financial lending.

Introduction and Motivation

Loan approval decisions significantly impact both financial institutions and borrowers. Incorrect approvals can increase financial risk and loan defaults, while incorrect rejections may prevent qualified applicants from receiving financial support.

Traditionally, loan approval decisions rely on manual assessment by financial officers who analyze applicant profiles, income levels, employment history, and credit records. As the number of applications increases, manual assessment becomes increasingly expensive and time-consuming.

Financial institutions must avoid risky borrowers when granting loans in order to safeguard their benefits, but determining which borrower is a qualified applicant or a risky applicant is challenging. As a result, it takes a lot of time and resources to filter applications, which could still lead to selection errors.

Machine learning models can serve as effective decision-support tools by providing an initial assessment of loan applications before they undergo manual review by financial officers. Rather than replacing human judgment, these models can help prioritize applications, improve consistency in the evaluation process, and reduce the time required to process large volumes of loan requests.

In this project, loan approval prediction is formulated as a supervised binary classification problem, where the objective is to distinguish between approved and rejected loan applications based on applicants' demographic, financial, and credit-related information and predict loan application status for new applicants. Given the class imbalance in the dataset, particular emphasis is placed on identifying approved applicants as reliably as possible while minimizing incorrect predictions. This approach enables the models to support lending decisions using historical data while maintaining a balance between predictive performance and practical applicability.

Accurate loan approval prediction has important practical implications for financial institutions. Improving the consistency and efficiency of the screening process can help reduce operational costs, support better risk management, and assist loan officers in making more informed decisions. Although machine learning models cannot eliminate financial risk, they can provide valuable data-driven insights that complement traditional lending practices and contribute to more efficient credit evaluation.

Related Work and Background

Machine learning has been widely adopted for loan approval and credit-risk prediction because it enables financial institutions to identify patterns in historical lending data that are difficult to capture using traditional rule-based systems. Numerous studies have shown that applicant credit history, income, existing debt, and loan characteristics are among the most influential factors affecting lending decisions [1], [2]. In particular, previous credit defaults have consistently been reported as one of the strongest predictors of loan approval and default risk, often outweighing demographic characteristics or employment information [3]. This agrees with the exploratory analysis in the dataset, where **previous_loan_defaults_on_file** emerged as the single most informative feature, motivating an additional comparison of models trained with and without this variable.

Several supervised learning algorithms have been successfully applied to loan approval prediction. Logistic Regression is frequently used as a baseline because it is computationally efficient, interpretable, and provides coefficients that help explain the contribution of individual features [4]. Support Vector Machines (SVMs) have been shown to perform well on non-linear classification problems by constructing decision boundaries that maximize class separation [5]. Ensemble methods, including Random Forest and Gradient Boosting, generally achieve higher predictive performance on tabular financial data because they can model complex feature interactions while reducing overfitting through the combination of multiple decision trees [6], [7]. These characteristics motivated their inclusion in this study alongside simpler baseline models and Gaussian Naive Bayes, allowing both predictive performance and interpretability to be compared across different modelling approaches.

Another challenge commonly reported in loan approval datasets is class imbalance, where approved and rejected applications are not equally represented [8]. Rather than relying solely on overall accuracy, recent studies recommend evaluation metrics such as Precision-Recall Area Under the Curve (PR-AUC), F1-score, and precision-recall analysis because they better reflect performance on the minority class [8], [9]. Following these recommendations, the project uses the F1-score of the approved class both during hyperparameter tuning and to set the final decision threshold, ensuring that model selection focuses on the prediction task, which is of greatest practical importance.

Model interpretability has also become increasingly important as machine learning systems are incorporated into financial decision-making. While explainability methods such as SHAP and LIME are widely used in the literature, they introduce additional computational complexity and were not required for the objectives of this project. Instead, permutation feature importance is used, a model-agnostic technique that measures the decrease in predictive performance when the values of an individual feature are randomly shuffled. This approach provides a consistent way to compare feature importance across different models while remaining straightforward to interpret. By combining

established classification algorithms, careful validation, and permutation importance analysis, this study aims not only to compare predictive performance but also to understand which applicant characteristics most strongly influence loan approval decisions.

Dataset Description

The dataset contains 45,000 loan applications, each described by 13 features and one label used for supervised binary classification. The columns record the applicant's personal details, credit history, requested loan, and financial situation, with each row representing one applicant. The data is a synthetic, enriched version of a standard credit-risk dataset [2], and it is complete with no missing values and no duplicate rows.

The label `loan_status` indicates the lender's decision: 1 if the application was approved and 0 if it was rejected. An approved applicant is assumed to be a good risk and a rejected one as a bad risk. This reading is not a verified record of whether a loan was repaid but an interpretation of the lender's past decision. The model's job is to identify the positive class, which consists of the approved (good risk) applicants.

The performance of the model is judged mainly by how well the model catches the good-risk or qualified applicants, not by overall accuracy. In this case, the label is imbalanced, with around 78% of applicants read as bad-risk (rejected) and 22% as good-risk (approved). Knowing this, a model that labelled every applicant a bad risk would still reach 78% accuracy, but it would reject every good customer, leading to bad performance. Because the good-risk class is the minority, it is the harder of the two to identify, and it is the focus of the project.

Every applicant with a previous loan default on file is a bad risk, without exception. Applicants with no default on file are split almost evenly between the two classes (rejected and approved). This single feature already resolves nearly half the cases, so the real learning challenge lies in the applicants with no recorded default. For this reason, the model is trained both with and without this feature.

Preprocessing and Exploratory Data Analysis

Before modelling, the dataset was studied in order to understand each feature and to identify problems that could affect the models. The data has no missing values and no duplicate rows. Every column varies, so nothing was dropped for being constant.

As seen in Figure 1, the target is imbalanced, with about 78% bad risk and 22% good risk (covered in the dataset description above).

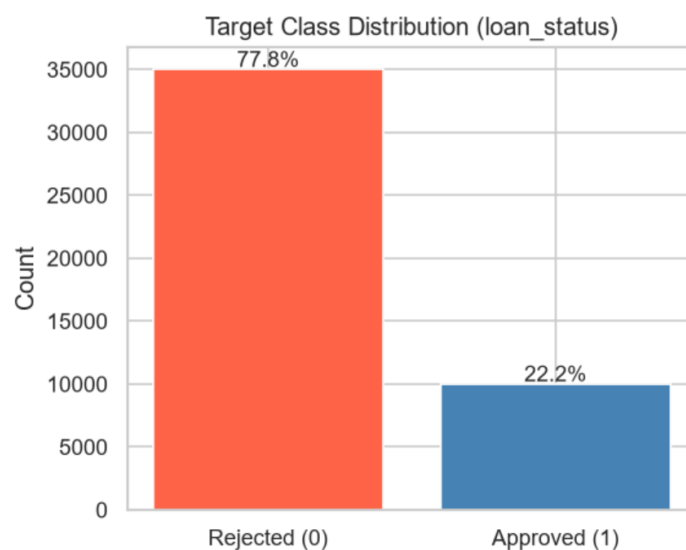


Figure 1. Class distribution of the target.

Bad-risk applicants form roughly 78% of the data and good-risk applicants 22%.

The categorical feature distribution was analyzed for the categorical columns: gender, education, home ownership status, loan intent, and previous loan defaults on file as shown below in Figure 2. It can be seen that male and female applicants are almost equally present in the dataset, with slightly higher number of male applicants. In terms of person education, bachelor's students are the most predominant one in the dataset and for home ownership status, renting and mortgage are the predominant ones with a select amount of people owning a house. For loan intent, all of the categories are spread out across the dataset but the primary loan intent is for education purposes. Lastly, for previous loan defaults on file, the number of people with and without defaults are almost split evenly with people with loan defaults being slightly higher.

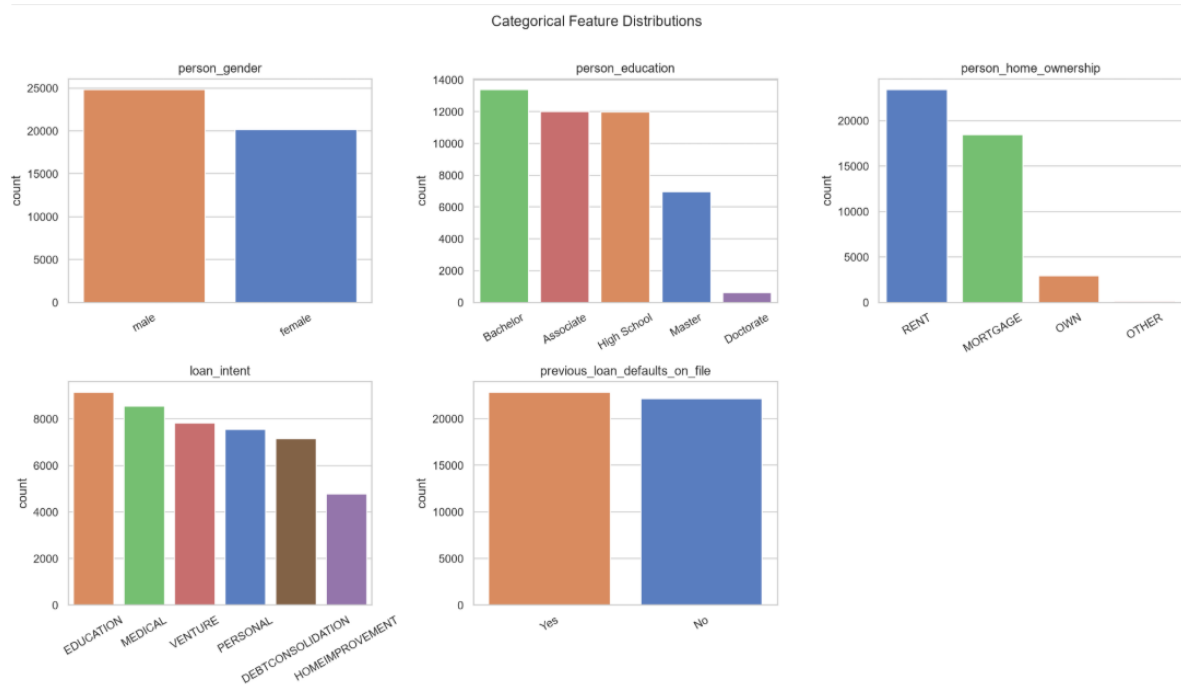


Figure 2. Distribution of categorical features in the loan approval dataset

Shows the frequency of each category across key demographic and financial variables.

Three numerical features needed attention. Two of them, `person_income` and `person_emp_exp`, were both heavily right-skewed, with a raw skew of 34.1 and 2.6, respectively. That is normal for income and work experience. Any feature with an absolute skew of 1 or more was flagged for a log transformation (Figure 3) and was further tested to determine if the absolute skew reduction after `log1p` transformation would be significant. After testing, `person_income` and `person_emp_exp` were chosen as `log1p` candidates since they showed significant skew reduction. Hence, a log transformation (`log1p`) was applied to both. This brought their skew close to zero (0.2 and -0.1, respectively) and stopped a few very large values from dominating the scale-sensitive models (the SVM and the linear model) (Figure 4).

The third feature was `person_age` since it has a few implausible values, with a maximum of 144 years, though only nine rows go above 80. Instead of removing these rows, `person_age` was capped at 80. That limit was chosen from the Canadian life expectancy of about 82.1 years [10].

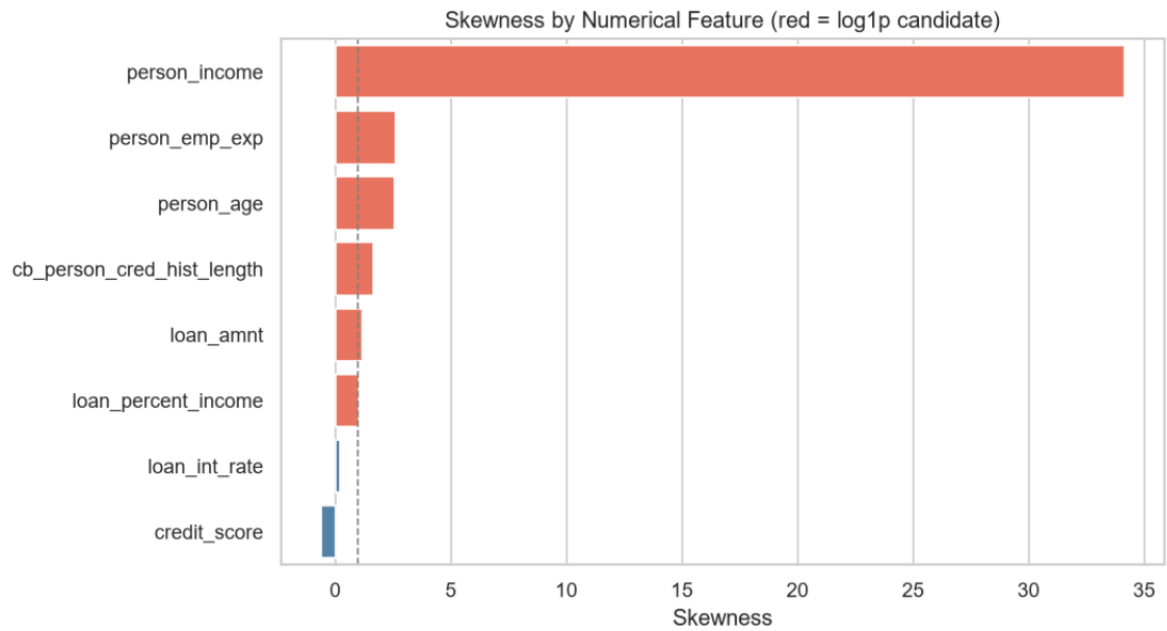


Figure 3. Skewness of each numerical feature.

Only person_income and person_emp_exp are selected for log transformation.

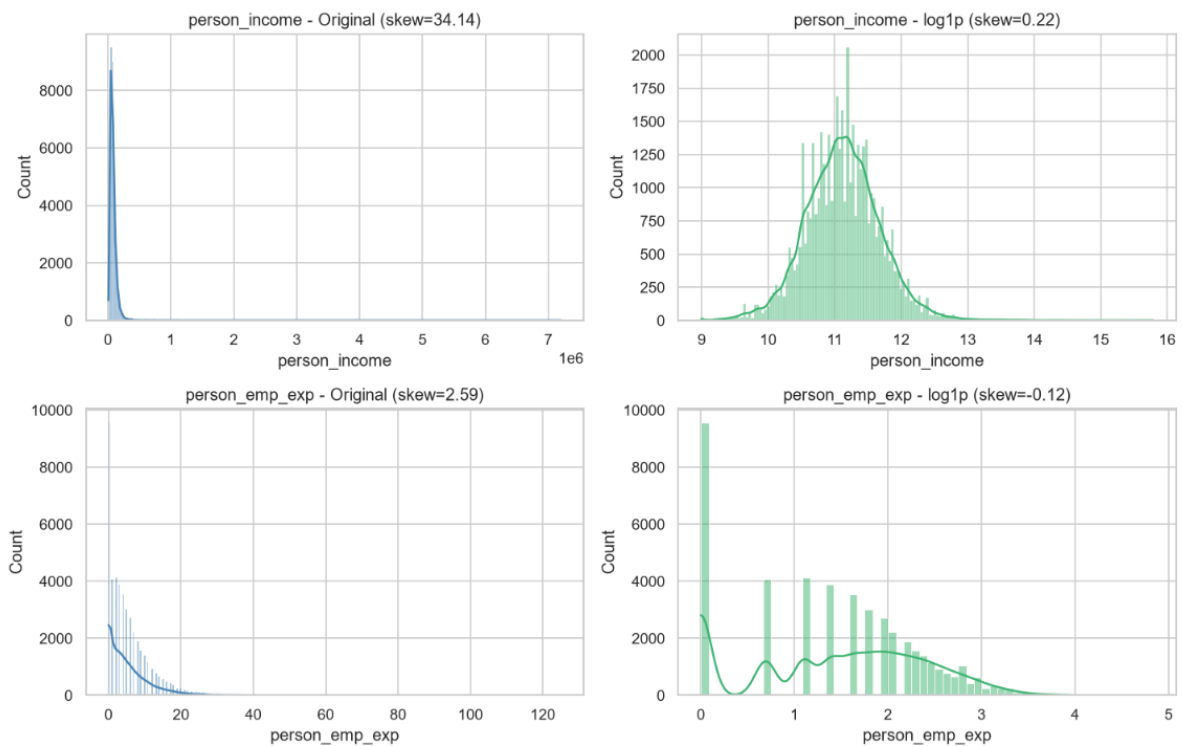


Figure 4. Distribution of person_income and person_emp_exp before and after the log transformation.

The raw values are heavily right-skewed, while the transformed values are close to symmetric.

The correlation analysis showed a cluster of related features (Figure 5). The `person_age`, `person_emp_exp`, and `cb_person_cred_hist_length` features are all strongly correlated, up to 0.95. This makes sense since older applicants tend to have more work experience and a longer credit history. This was not treated as an automatic reason to drop features. Tree-based models can pick whichever of the correlated features is most useful and ignore the rest, so they are not hurt by correlated inputs, and all three are kept. For the linear model, correlated inputs can inflate coefficient variance, so dropping `person_age` in favour of `cb_person_cred_hist_length` was considered, but the feature set was kept whole since the tree ensembles are unaffected and are expected to be the stronger models. `loan_percent_income` was also checked against loan amount and income because `loan_percent_income` is derived from the former two, but it was not strongly correlated with either, so it was kept.

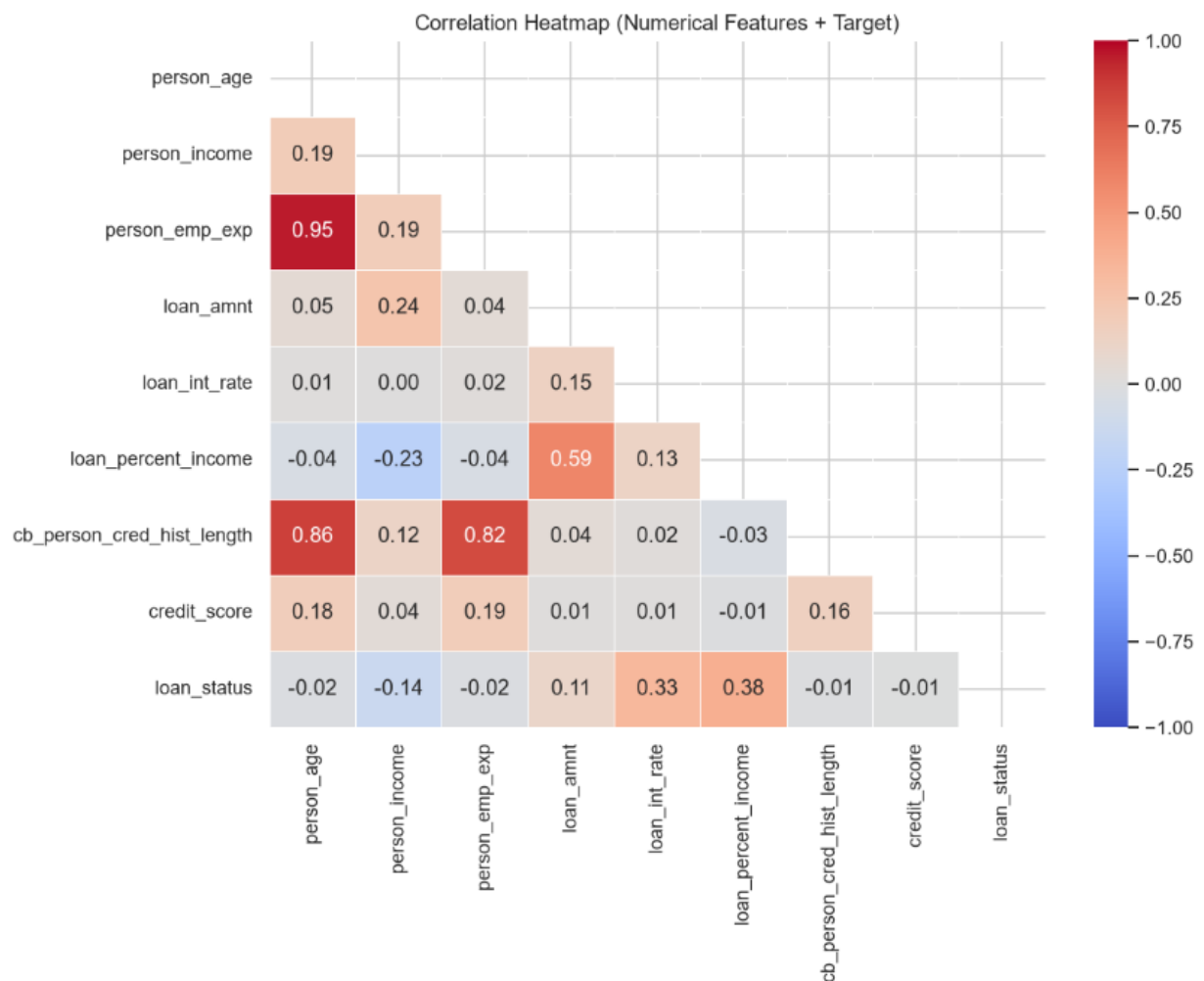


Figure 5. Correlation matrix of the numerical features.

person_age, person_emp_exp, and cb_person_cred_hist_length form a strongly correlated cluster.

The relationship between each feature and the target was examined next (Figure 6).

`loan_percent_income` and `loan_int_rate` separate the two classes most clearly, with bad-risk applicants tending to have higher values. Features like credit score separate them less than expected. This suggests the loan size relative to income, and the offered interest rate, carry more importance than any single applicant attribute.

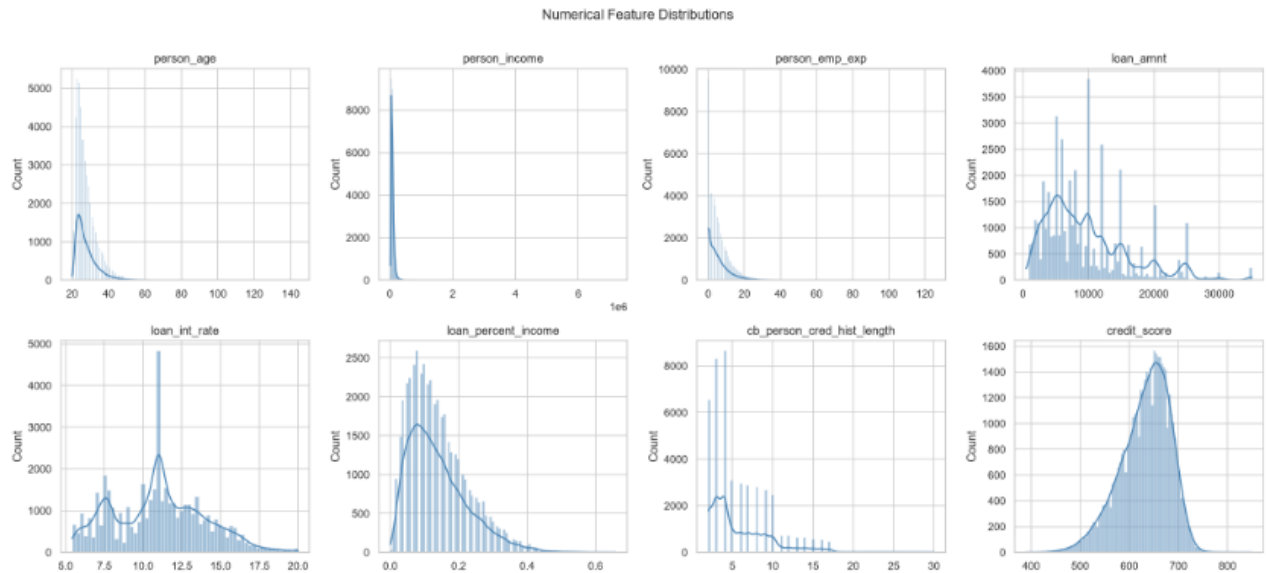


Figure 6. Distribution of each numerical feature by class.

`loan_percent_income` and `loan_int_rate` show the clearest separation between good and bad risk.

The same relationships are shown in more detail in Figures 7 and 8. The box plots in Figure 7 confirm the medians and spreads behind this, with bad-risk applicants tending toward higher `loan_int_rate` and `loan_percent_income`. The violin plots in Figure 8 add the shape of each distribution, showing not just where the two classes differ on average but where their values pile up and overlap. The overlap is clearest for features like `person_age` and `credit_score`, which is why these alone do not separate the classes well, while `loan_int_rate` and `loan_percent_income` show clearer separation between the two groups.

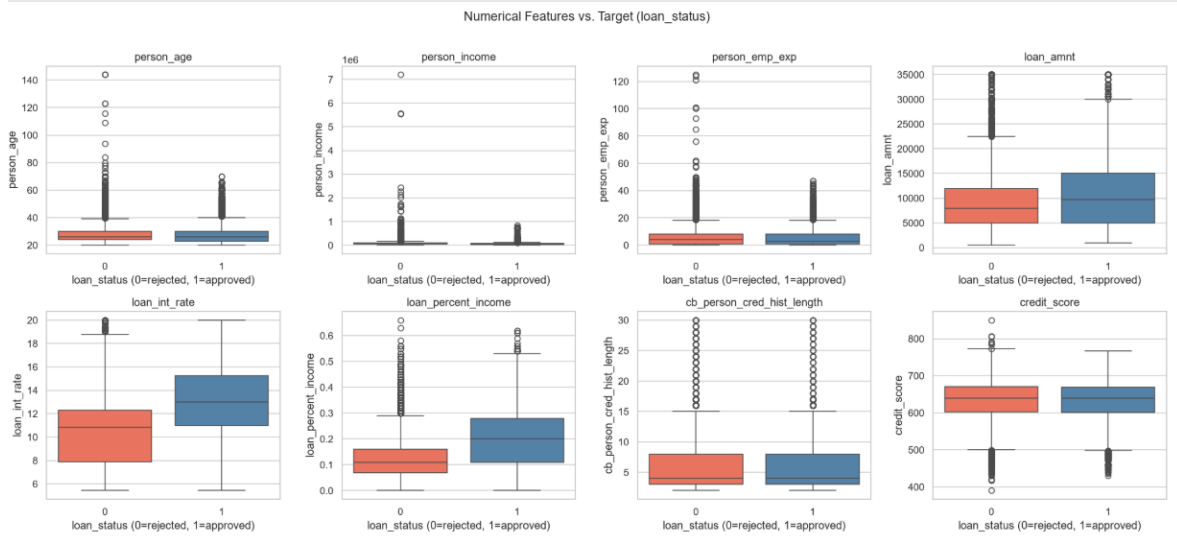


Figure 7. Numerical features versus the target, shown as box plots by class.

Bad-risk applicants tend toward higher interest rates and loan-to-income ratios.

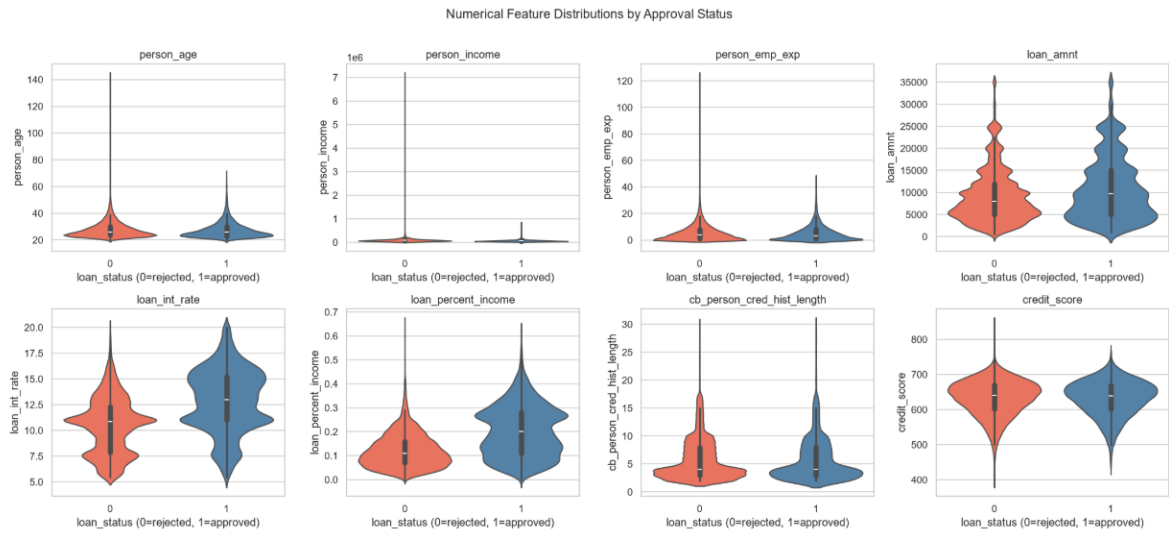


Figure 8. Distribution of each numerical feature by class, shown as violin plots.

The width of each shape shows where values concentrate, making the overlap between the two classes visible for features such as age and credit score.

The categorical features were examined the same way, by comparing approval rates across categories (Figure 9). Home ownership and loan intent have a clear effect: renters and certain loan purposes are approved at very different rates. It is also observed that gender and education both have only a small difference between approval rates, showing little effect. The label records the lender's past decisions,

not repayment outcomes. So these patterns show how applications were treated historically, and they are not read as a direct measure of risk.

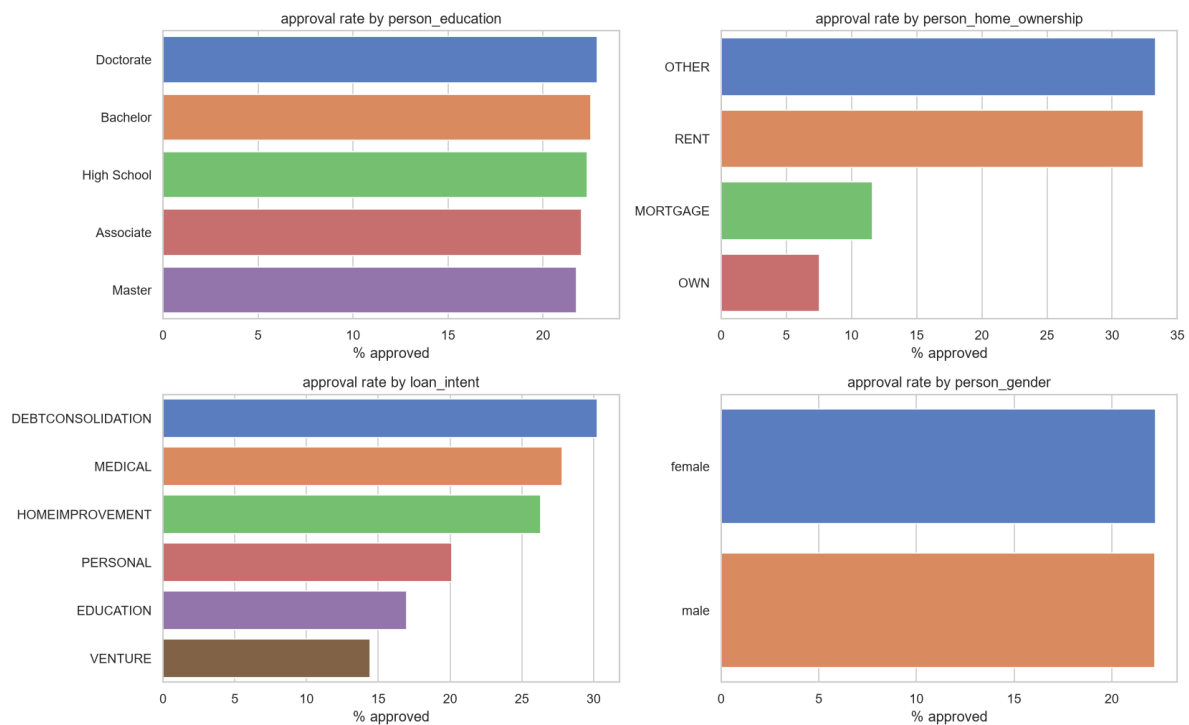


Figure 9. Approval rate across the categorical features.

Home ownership and loan intent show the largest differences between groups.

The dominance of `previous_loan_defaults_on_file`, noted earlier, is shown in Figure 10. This is not data leakage. The field reflects the applicant's prior credit history, which is known when they apply and is not the outcome of the current loan. The feature is kept, but because it resolves nearly half the dataset on its own, the models are also trained without it. This is to see how much of their performance depends on this one feature.

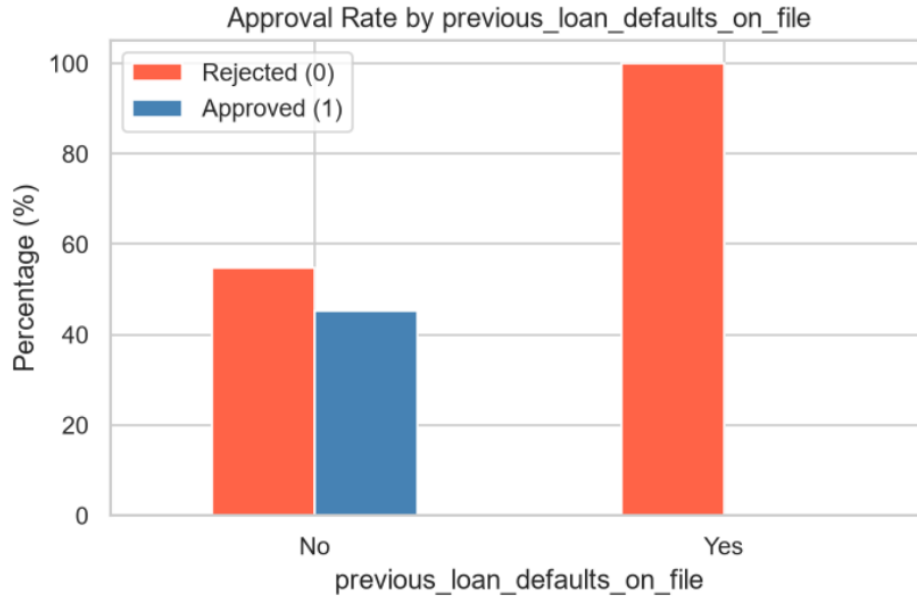


Figure 10. Class distribution by previous_loan_defaults_on_file.

Every applicant with a prior default falls in the bad-risk class, while applicants with no default are split almost evenly.

The preprocessing pipeline applies the cleaning steps above (capping age, log-transforming income and experience, scaling, and encoding) in a leakage-safe way. Each step that learns from the data, such as the scaler computing a mean and standard deviation, is fit on the training set only, and the test set is then transformed using those same training values, so no test-set information influences the preprocessing. Additionally, person_education is ordinally encoded to keep its natural order (High School < Associate < Bachelor < Master < Doctorate), while the remaining categorical features (gender, home ownership, loan intent, and previous defaults) are one-hot encoded. This produces a 23-column feature matrix, and a check confirms that the target is not among the features. Two engineered features are also defined: employment experience over adult years and credit history over adult years. They use row-wise arithmetic only, so they add no leakage and can be created before the split. These features are used in the feature-set comparison reported in the error analysis.

The data is split 80/20, stratified on the target label, so the 78/22 balance holds in both sets. The test set is held out for the final evaluation only. All tuning happens inside the training set.

Methodology and Models

The model predicts `loan_status` (1 if the lender approved the application, 0 if it rejected it). The approved, good-risk class is the minority, about 22% of the data. So the models are built to catch this smaller class well, not to score high on overall accuracy. As noted in the data exploration, the label is the lender's past decision, not a record of repayment, so the model learns who tends to get approved. Any talk of credit risk is a reading of that decision, not a direct measure of default.

The preprocessing and the 80/20 split described above run inside one scikit-learn pipeline and lets each model be saved as one object that takes a raw application in and returns a prediction.

Several models are compared, from simple to complex. The models used are listed below:

1. Dummy Classifier (Always predicts the majority class)
2. Logistic Regression - Baseline Model
3. SVM (RBF-kernel)
4. Random Forest
5. Gradient Boosting

A dummy classifier that always predicts the majority class is implemented first and is the floor every real model must beat. Logistic regression is a simple linear model whose coefficients are easy to read, which makes it a clear reference point for the more complex models. This is why this was chosen as our **baseline model**. A support vector machine (SVM) with an RBF kernel adds a model that can draw a non-linear boundary. RBF kernel was chosen as opposed to linear or polynomial kernels because RBF kernels perform better for handling complex non-linear boundaries present in our dataset. Standardizing the numerical features matters most for this model and the linear one, since both react to scale. Gaussian Naive Bayes is a fast probabilistic model included as a baseline. The one-hot encoded columns break its independence assumption, so it is expected to trade precision for very high recall. Random forest and gradient boosting are the two ensembles, gradient boosting being a histogram-based gradient boosted tree. These ensembles handle mixed feature types and non-linear interactions well on tabular data, and gradient boosting is expected to be the strongest model [11].

The class imbalance is handled with model weights and the decision threshold, not by resampling. Each model that supports it is trained both with and without balanced class weights, and cross-validation picks the better option. The decision threshold is also moved off the default 0.5 and is tuned for each model, so the model catches the minority good-risk class more reliably. The search procedure, the scoring metric, and the threshold rule are described in the following two sections, while the full results are in the model comparison.

Two choices support the analysis later: training each model with and without the dominant feature and interpreting the stronger models with permutation importance. `previous_loan_defaults_on_file` resolves almost half the dataset on its own, so every model is also trained without it. This shows how much each model leans on that one feature and how well the rest of the profile holds up without it. Permutation importance measures a feature's value by how much performance drops when its values are shuffled, which gives a model-agnostic view of what drives the predictions [12].

Validation and Hyperparameter Tuning Strategy

The tuning strategy involves using the training set in a k-fold stratified cross-validation, meaning the training set is split into k-folds. Each fold is used once for validation while the other folds train the model, so every row is validated exactly once. Stratifying keeps the class balance in every fold, so no fold is short of the minority good-risk class. The test set takes no part in this. It is held out until the final evaluation, so nothing learned during tuning has ever seen it, to avoid data leakage.

The cross-validation runs inside the pipeline. The scaler and encoders are refitted on each fold's training portion, and the validation fold is then transformed with those values. This stops information from the validation rows leaking into the fitted transforms and inflating the scores.

During each model's first run, the grid search trains every combination of the listed hyperparameters under cross-validation and attempts to keep the best combination of hyperparameters. Comparing the tuned result against the untuned baseline shows the difference between the untuned and tuned results and how the hyperparameters affect the model's performance.

An imbalance-aware metric is used instead of accuracy [9]. Accuracy rewards a model for getting the large bad-risk class right while ignoring the small good-risk class, so it can hide poor performance on the class that matters. The F1 score balances precision and recall on the good-risk class, so it measures what the project cares about rather than rewarding the majority class. ROC-AUC is left out on purpose, since it can look optimistic on imbalanced data [8].

The decision threshold is tuned as a separate step once the model is fixed. Rather than assuming the default 0.5 cutoff is best, the threshold is swept across its range, and the value that gives the highest F1 score on the good-risk class is chosen. This threshold is selected on out-of-fold predictions on the training set, where each row is scored by a copy of the model that never saw it during training. Choosing it this way avoids the optimism of tuning the threshold on the same in-sample predictions the model was fit on, which would be inflated for the tree ensembles in particular.

The decision threshold tuning results for our chosen 5 models along with the graphs are listed below in Figure 11, 12, 13, 14 and Figure 15. Based on the out-of-fold threshold folding on training set and F1-score metric, the tuned thresholds for all models are listed below:

1. Gradient Boosting Tuned Threshold = 0.415
2. Random Forest Tuned Threshold = 0.572
3. Logistic Regression Tuned Threshold = 0.411
4. SVM-RBF Tuned Threshold = - 0.303
5. Gaussian Naive Bayes Tuned Threshold = 0.625

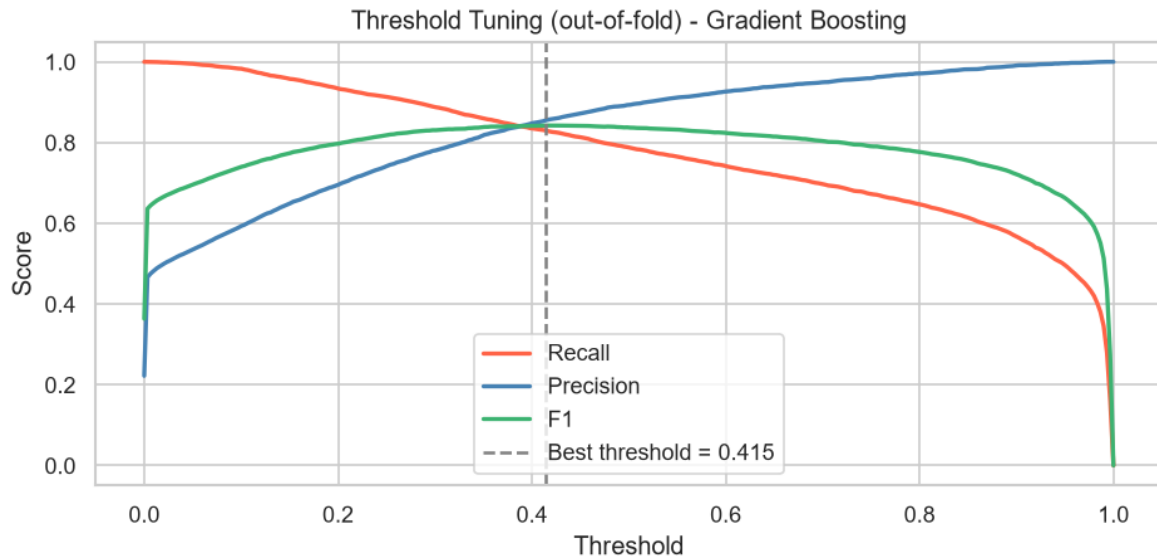


Figure 11. Threshold Tuning for Gradient Boosting (out-of-fold).

Precision, recall, and F1 on the good-risk class as the threshold sweeps; F1 peaks at 0.415 (dashed line).

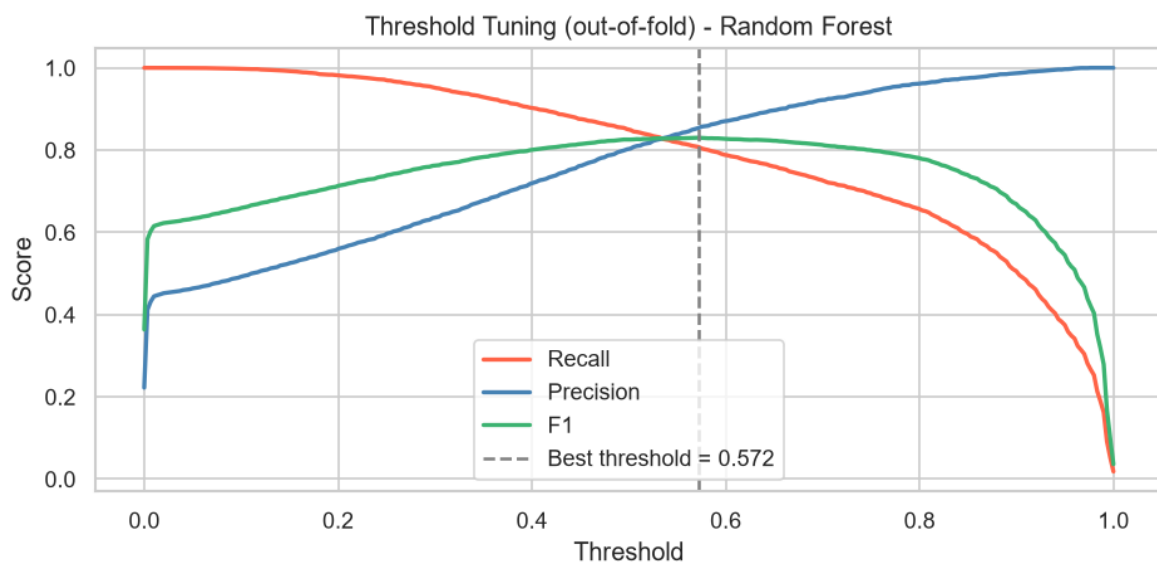


Figure 12. Threshold tuning for Random Forest (out-of-fold)

F1 on the good-risk class peaks at a threshold of 0.572 (dashed line).

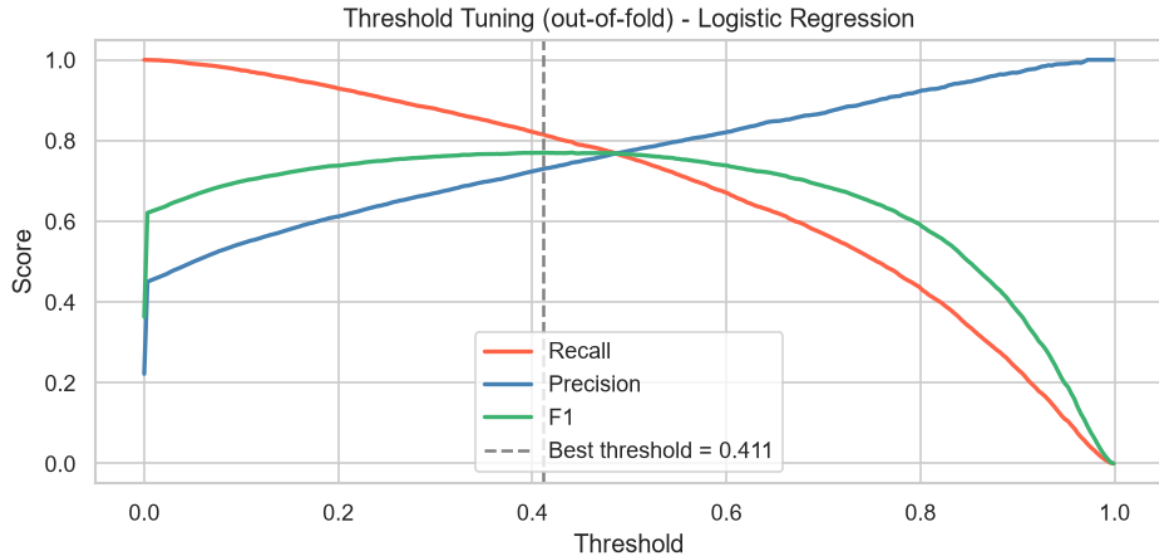


Figure 13. Threshold tuning for Logistic Regression (out-of-fold).

F1 on the good-risk class peaks at a threshold of 0.411 (dashed line).

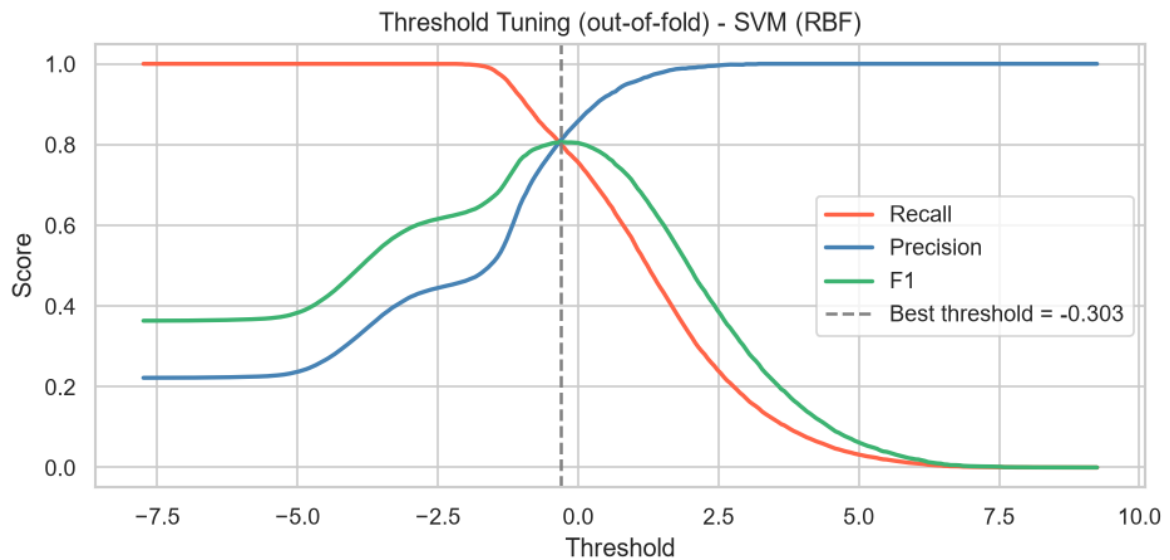


Figure 14. Threshold tuning for SVM (RBF) (out-of-fold).

Plotted on the decision-function scale rather than 0–1, so the best threshold is negative (−0.303).

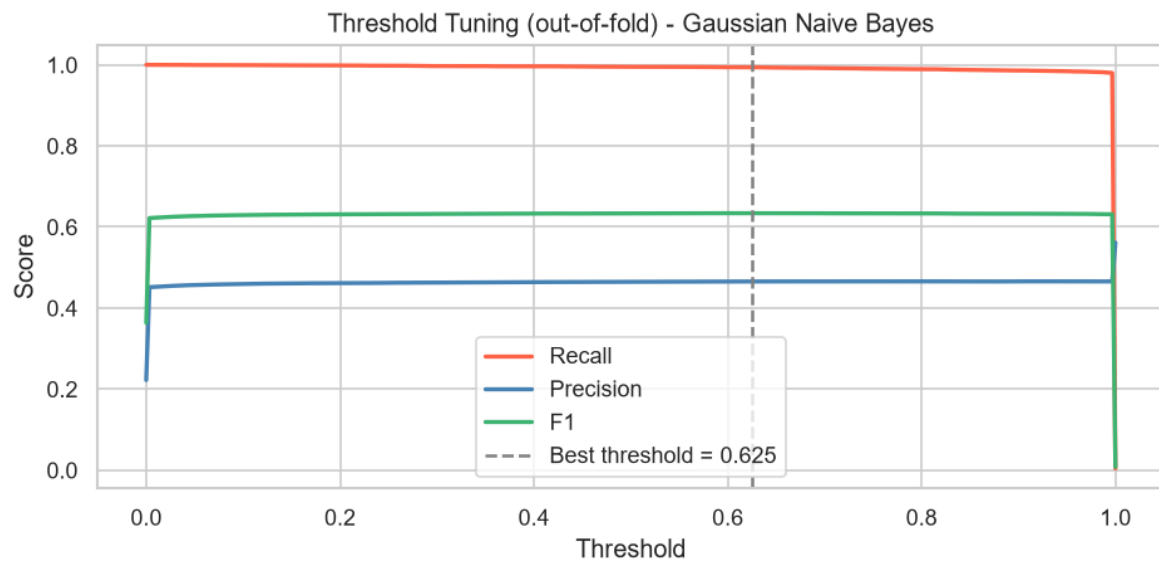


Figure 15. Threshold tuning for Gaussian Naive Bayes (out-of-fold).

F1 stays nearly flat and precision never exceeds ~ 0.45 , so no threshold recovers good precision; best threshold 0.625.

Experimental Setup

The tuning strategy is set up to use 5-fold stratified cross-validation, splitting the training set into five equal folds. One fixed random seed is used for the split, the cross-validation folds, and every model in order to reproduce the procedures. The fixed seed and the project paths are maintained in a single python file named “utils” to improve codebase reproducibility and organization. For each of the notebooks except EDA, helper functions were written as python files under /src, which were used by each of the notebooks.

The grids are kept small and specific to each model (Table 1). Hyperparameter configurations are scored on the F1 score of the good-risk class, and the best configuration for each model is the one with the highest F1.

Model	Hyperparameters searched
Logistic Regression	regularisation strength C {0.001, 0.01, 0.1, 1, 10} l1_ratio {0, 0.5, 1} class weight {none, balanced}
Support Vector Machine (RBF)	regularisation strength C {0.1, 1, 10} kernel width gamma {scale, auto} class weight {none, balanced}
Gaussian Naive Bayes	smoothing var_smoothing {1e-9, 1e-8, 1e-7}
Random forest	number of trees {200, 400} maximum tree depth {none, 10, 20} class weight {none, balanced}
Gradient boosting	learning rate {0.05, 0.1} maximum tree depth {none, 6} maximum iterations {200, 400} class weight {none, balanced}

Table 1. Hyperparameter grids searched for each model.

Each value set lists the options the grid search tried for that hyperparameter; class weight "balanced" reweights the classes by inverse frequency, "none" leaves them unweighted.

Results and Model Comparison

Five classification models were evaluated to determine how well each model would perform. The same test data was utilized during the testing phase for all the models; this is to eliminate bias and ensure true comparability of the models.

Model performance was evaluated using precision, recall, accuracy, balanced accuracy, and F1-score, with PR-AUC also reported. Multiple metrics are used here because they show the different types of errors the models can make. The F1-score gives a balanced trade-off between precision and recall on the approved class, which is the class of interest, and is the metric the models are selected on.

The graph below shows the PR-AUC curve for all evaluated models on the test set.

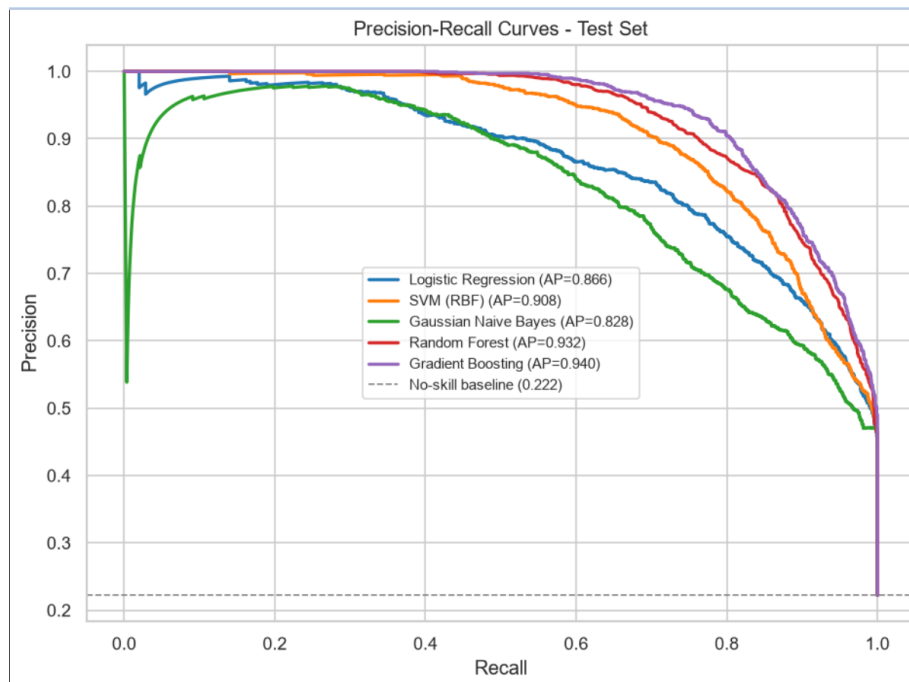


Figure 16. PR-AUC curve for all selected models.

Gradient boosting achieves the highest PR-AUC, demonstrating the best trade-off between precision and recall across all decision thresholds

The above figure illustrates that gradient boosting has superior performance across a range of thresholds; however, the final model selection was based on the F1 score, that is, after threshold tuning using out-of-fold predictions on the training set, the model with the highest F1 score (best out-of-fold) threshold is the best model to utilize.

The result from running the various models on the test data as per the table below (Table 2) shows that gradient boosting performs well on this classification problem with the highest F1 score of 0.8458. Naive Bayes performed the least; this is because it assumes independence among each feature. In broad benchmarks, Naive Bayes consistently ranks among the weaker classifiers [13]. Here its poor score comes from its core assumption that the features are independent, which is violated by the correlated and one-hot-encoded features in the dataset.

Model	Threshold	Accuracy	Balanced Acc	Precision	Recall	F1 (Primary Metric)	PR-AUC
Gradient Boosting	0.4147	0.9321	0.8985	0.8538	0.8380	0.8458	0.9402
Random Forest	0.5719	0.9296	0.8885	0.8610	0.8145	0.8371	0.9318
SVM (RBF)	−0.3029	0.9168	0.8772	0.8170	0.8060	0.8115	0.9084
Logistic Regression	0.4109	0.8954	0.8667	0.7406	0.8150	0.7760	0.8659
Gaussian Naive Bayes	0.6254	0.7498	0.8374	0.4702	0.9950	0.6386	0.8280
Dummy Baseline	N/A	0.7780	0.5000	0.0000	0.0000	0.0000	0.2220

Table 2. Test Set Results (sorted by F1).

The SVM threshold is reported on its decision-function scale rather than as a probability, which is why it is negative.

Selected primary model from the test set: Gradient Boosting (F1=0.8458)

When accessed on the independent test data, each of the models used exhibits distinct learning characteristics. Overall, the ensemble methods outperform the simpler ones, with gradient boosting at the top, followed by random forest and the SVM, then logistic regression, and Gaussian Naive Bayes at the bottom. The F1-score ranking in Table 2 is the basis for this comparison.

Although nonlinear boundaries improve model performance, Support Vector Machine (RBF Kernel) produces some good results but performs worse than ensemble methods. This indicates that ensemble-based models are far more appropriate for the dataset utilized for this project.

Logistic regression is the weakest of the trained models apart from Naive Bayes, and as the table illustrates, its precision (0.7406) is lower than that of the more complex models, reflecting its difficulty with non-linear patterns

Additionally, table 2 demonstrates that Naive Bayes has the highest recall score but a much lower precision and it's accuracy is lower than our dummy baseline model. This shows us that Naive Bayes has a stronger tendency to overpredict the positive class and proves once more that this model assumes that there is no dependence between the features, but that is not true with data sets that contain financially correlated data. Hence, a very poor model to utilize for such a machine learning problem.

Overall, the results from the test demonstrated that ensemble learning methods are the most reliable when it comes to classification problems with high-dimensional nonlinear data and when the costly errors fall on the minority class. Ensemble approaches consistently outperform linear models, according to earlier research on tabular financial datasets [7]. Additionally, the performance of tree-based ensembles has been further enhanced by the introduction of highly scalable gradient boosting systems [14], which has increased the performance gap between these techniques and linear methods.

Based on test-set results and evaluation metrics, the performance and behaviour of the assessed machine learning models are compared below.

- **Gradient Boosting** did overall well. It works by making a series of trees, with each tree fixing the mistakes of the one before it. This way the model can really get a handle on relationships between things like credit score, loan-to-income ratio, and past defaults. Linear models just cannot do this well.
- **Random Forest** was a bit behind. Still did a good job. It uses a technique called bagging, where it makes a lot of decision trees from different sets of data and features. This helps make the model more stable. It does not keep fixing its mistakes like gradient boosting does, so it is not as good in the end.

- **SVM with an RBF kernel** worked okay. It takes a lot of computer power. It can handle boundaries in a high-dimensional space due to kernel transformation. However, it is very sensitive to how the features are scaled, and it gives a raw score instead of a probability, which makes it harder to understand. So you can still adjust the threshold because it is applied right to the output of the decision function.
- **Logistic Regression** is the baseline model to compare to. It assumes that the features and the target variable are related in a line, so it can only separate the classes with a straight line. Since loan approval is influenced by relationships between features, it does not do as well. On the one hand, it is the easiest model to understand because the coefficients show exactly how much each feature matters.
- **Gaussian Naive Bayes** did the worst in terms of precision even though it was good at recalling things. This is because it assumes that the features are independent, which is not true in this case. There are correlations between the features, especially because some of them are variables that were one-hot encoded. As a result, the model tends to overestimate the probability of approval, and gradient boosting is still the model.

The table below summarizes the differences between the used models, highlighting their strengths and costs.

Model	Interpretability	Complexity	Training Cost	Notes
Logistic Regression	High - coefficients directly readable	Low	Fast (seconds)	Linear boundary; struggles with non-linear patterns
SVM (RBF)	Low - kernel space not interpretable	Medium	Slow (minutes on 36K rows)	Strong margin-based classifier; no probability output by default
Gaussian Naive Bayes	Medium - per-feature likelihoods	Very low	Fastest	Independence assumption violated by OHE features; high recall but low precision
Random Forest	Medium - feature importances available	High	Moderate (400 trees)	Robust; averages many trees; less prone to overfitting than single DT
Gradient Boosting	Low - ensemble of trees	High	Moderate	Sequential boosting captures complex patterns; selected only if it has the highest test-set F1

Table 3: Model strength and weaknesses summary

Trade-off Comparison of all assessed models emphasizing computational cost during training, interpretability, and model complexity.

Error analysis and Qualitative Discussion

Beyond the headline metrics, this section interprets the results qualitatively. It breaks down the errors made by the chosen model, gradient boosting, compares the error behaviour of all five models, examines which features drive the predictions, and discusses where the model is most likely to fail. The dataset and model limitations, and directions for improvement, are covered in the Limitations and Future Work section.

1. Error Breakdown

Despite outperforming all other models, gradient boosting still generates two types of classification errors: false positives and false negatives. With the approved class as positive, a false positive is an applicant the model predicts as approved who was actually rejected, and a false negative is an applicant the model predicts as rejected who was actually approved.

Because the label is the lender's historical decision and not a repayment outcome, neither error maps directly to a financial loss; both are reported here so the model's behaviour near the decision boundary can be understood.

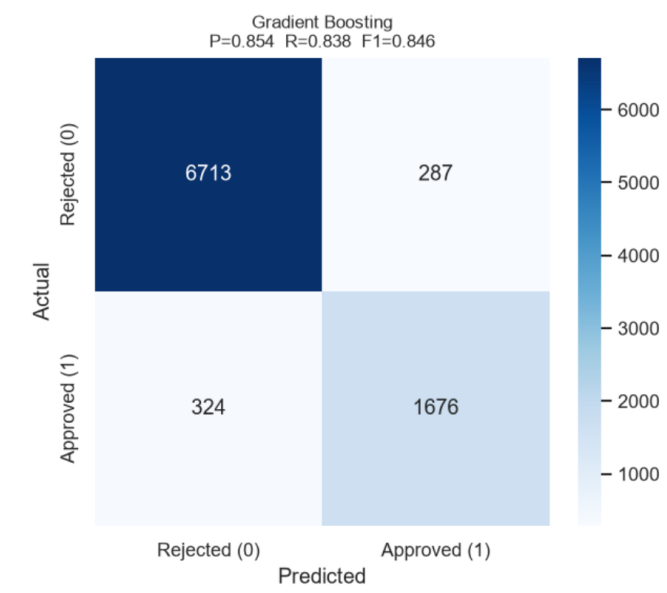


Figure 17. Confusion Matrix for Gradient Boosting on Test Set

Counts of true positives, true negatives, false positives, and false negatives in the loan approval decision.

The illustration shows that while most predictions are correctly classified, there are still some false positives and false negatives. It is more difficult to determine which class these applicants belong to because they are on or close to the decision boundary.

The confusion matrices for all models are given below in Figure 18.

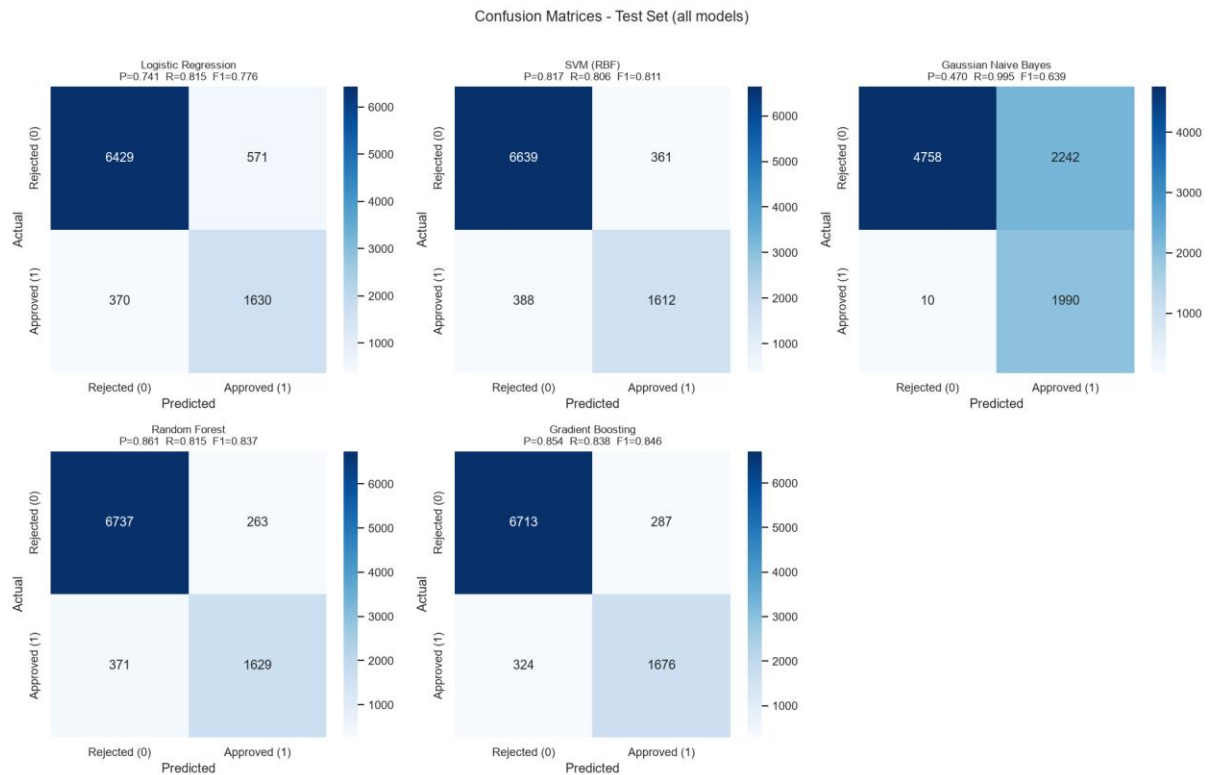


Figure 18. Confusion Matrices for all 5 models

Each matrix shows predicted versus actual labels, with the diagonal giving correct predictions (true negatives top-left, true positives bottom-right) and the off-diagonal giving the two error types.

From the above figure, we can see that Random Forest has the highest number of True Positive values, with Gradient Boosting coming closely in second. However, gradient boosting predicts True Negatives more accurately, with fewer false positives, which is what gives it the higher F1 and edges it ahead of random forest overall. The SVM follows a similar pattern to the two ensembles but makes slightly more errors of both types. Logistic regression makes more mistakes across the board, reflecting the limits of its linear boundary. Gaussian Naive Bayes stands apart since it labels almost every applicant as approved, so it has very few false negatives but a large number of false positives, which matches its near-perfect recall and very low precision in Table 2. This is why the two ensembles outperform the simpler models, as they handle the borderline applicants that the linear and probabilistic models cannot separate.

2. Patterns in Misclassification

To obtain a better understanding of the errors, the mean characteristics of true positives, true negatives, false positives, and false negatives for each feature were compared as follows in the table below.

	True Positive	False Positive	True Negative	False Negative
person_age	27.36	28.14	27.85	27.91
person_income	55914.97	70841.76	86023.37	71162.16
person_emp_exp	5.03	5.76	5.52	5.41
loan_amnt	11404.18	8821	9205.89	8232.29
loan_int_rate	13.12	13.34	10.34	11
loan_percent_income	0.22	0.14	0.12	0.13
cb_person_credit_history_length	5.78	6.03	5.91	5.84
credit_score	626.33	632.27	633.19	648.74

Table 4. Mean feature values by prediction outcome

Selected average of false positives and false negatives from specific features to illustrate decision boundary uncertainty.

The aforementioned findings demonstrate that misclassifications mostly take place in feature space borderline areas where class separation is less pronounced. The model is heavily dependent on certain characteristics, and when these characteristics clash (for instance, an applicant with a high income and good credit score but a history of loan defaults on file), the model becomes confused and makes some inaccurate predictions.

Figure 19 shows the feature interpretation plot that was created.

In comparison to certain other characteristics, gradient boosting gives priority to past loan default, income, the loan intent, and amount applied for, as well as credit score and age. Because the model may accept an applicant even if they have a poor credit score, there are some FPs and FNs. Based on the credit score feature, Table 4 shows that the mean credit score of false positives (632) is close to that of false negatives (649), indicating that applicants in this category have similar credit profiles and are therefore difficult to separate.

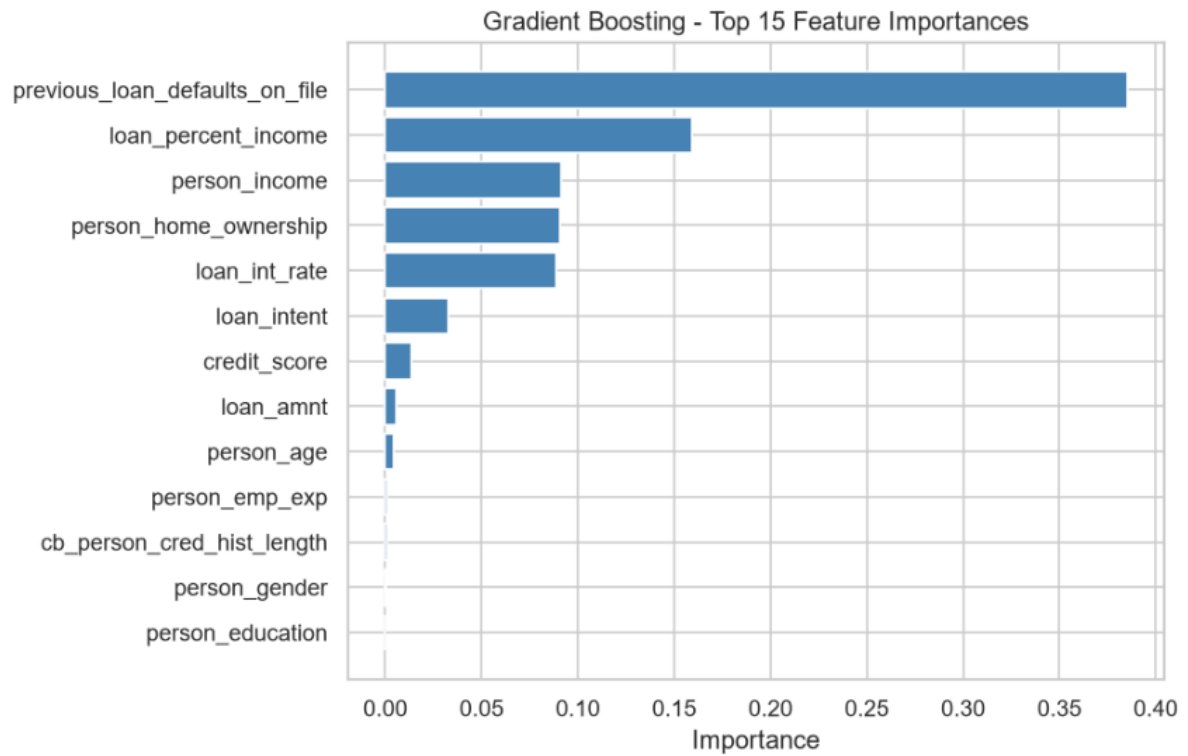


Figure 19. Feature Importance for Gradient Boosting

Plot illustrating each feature's relative contribution and importance to gradient boosting within the project's scope, with prior loan default being the most significant.

Demo or Usage Demonstration Description

To illustrate the practical use of our trained machine learning models, we opted for two different demo options:

1. An interactive Jupyter notebook
2. A Command-Line Script (CLI)

Jupyter Notebook:

A separate notebook was created to allow for sample applicants to be tested against the model. Included are three different sample applicants with varying input features to showcase different applicant scenarios. Their data can be modified in order to test against other desired scenarios.

Running the notebook via the Run All command will allow each code cell to be executed and the corresponding outputs to be displayed. If preferred, each notebook cell can also be executed individually, though in order, to see the individual steps.

The notebook loads two Gradient Boosting pipelines as chosen previously as the best-performing models, one with and one without **previous_loan_defaults_on_file**. The applicants are then passed through each model pipeline, and each pipeline outputs an approval probability, which is then converted to a binary decision of Approved or Rejected using each model's tuned thresholds. The compiled results of an example pair of applicants are shown in Figure 20. Figure 21 also shows an additional visual figure that shows the visual relationships between both pipelines and all applicants.

	Profile	Previous defaults	With feature P(Approved)	With feature Decision	Without feature P(Approved)	Without feature Decision	Probability difference (with - without)	Decision changed?
0	Applicant A	No	20.4%	REJECTED	11.3%	REJECTED	+9.1%	False
1	Applicant B	No	100.0%	APPROVED	99.9%	APPROVED	+0.1%	False
2	Applicant C	Yes	0.0%	REJECTED	9.4%	REJECTED	-9.4%	False

Figure 20. Prediction results for a sample pair of applicants through both Gradient Boosting pipelines.

Each pipeline outputs an approval probability, converted to an Approved or Rejected decision using its own tuned threshold.

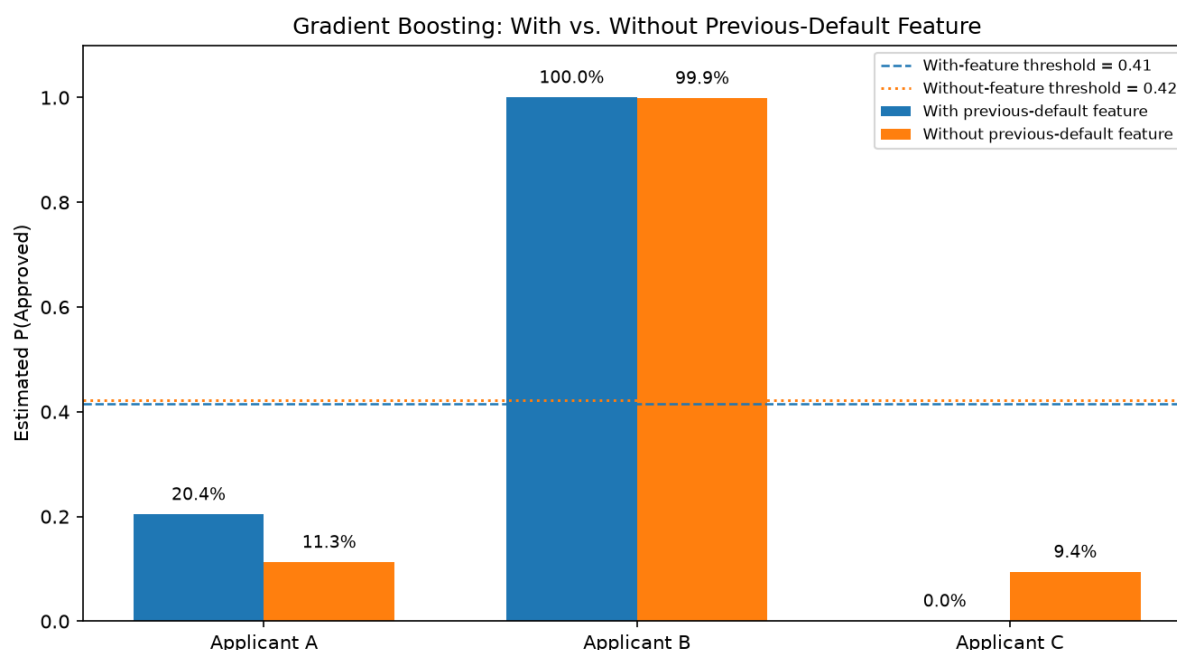


Figure 21. Approval probabilities for all sample applicants under the with- and without-prevdef pipelines.

Probabilities are plotted against each model's tuned threshold; including `previous_loan_defaults_on_file` raises the probability for applicants with no prior default and lowers it for those with one.

We can see that, including **`previous_loan_defaults_on_file`** shifts each applicant's estimated approval probability upward for those with no prior loan defaults, downward for those with them, while the rest of the profile is identical. The bar chart plots these probabilities against each model's own tuned thresholds. Probability change without a decision change still matters because it reflects how much confidence the previous default feature carries.

Due to the nature of the **`previous_loan_defaults_on_file`** input feature, an additional test was added where each applicant profile was re-run into the pipeline with constant input values apart from the aforementioned one, which was flipped for each applicant to see the effects of the input feature on the model. The results of the re-run is shown below in Figure 22.

Profile	With feature P(No defaults)	With feature P(Yes defaults)	With feature delta (Yes - No)	With feature Decision (No -> Yes)	With feature decision flipped?	Without feature P(No defaults)	Without feature P(Yes defaults)	Without feature delta (Yes - No)	Without feature Decision (No -> Yes)
0 Applicant A	20.4%	0.0%	-20.4%	REJECTED -> REJECTED	False	11.3%	11.3%	+0.0%	REJECTED -> REJECTED
1 Applicant B	100.0%	0.1%	-99.9%	APPROVED -> REJECTED	True	99.9%	99.9%	+0.0%	APPROVED -> APPROVED
2 Applicant C	36.1%	0.0%	-36.1%	REJECTED -> REJECTED	False	9.4%	9.4%	+0.0%	REJECTED -> REJECTED

Figure 22. Effect of flipping `previous_loan_defaults_on_file` from No to Yes for all three applicants.

All other inputs are held constant. The probability drops sharply in the model that uses the feature, while the model trained without it is unchanged.

We can see from the above table that, flipping only `previous_loan_defaults_on_file` from No to Yes sharply lowers the approval probability in the model that uses it. Applicant B's result is interesting because flipping the applicant's previous default on file from No to Yes predicted the applicant as rejected and showed a sharp approval probability decline of almost ~99%. This is strong evidence of the 'previous_default_on_file' feature's influence, though it shows association, not causation. This evidence confirms that the trained model learned an association between previous-default history and historical loan approval. The model trained without the feature is unaffected, confirming that it ignores default history entirely.

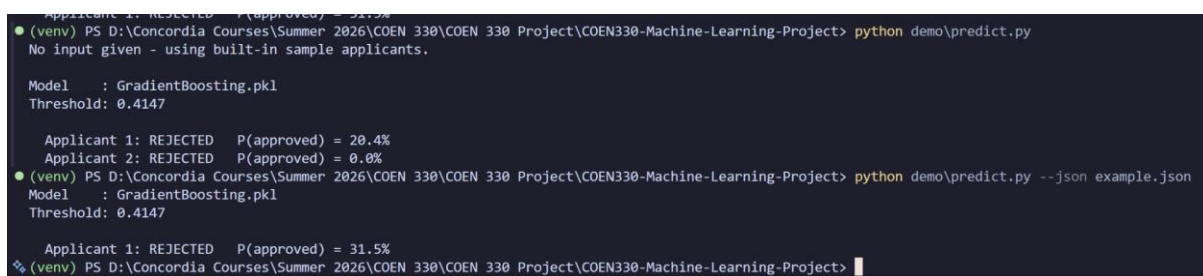
Command-Line Script (CLI):

A command-line script file "predict.py" was created that loads the model and predicts on sample input. Users will be able to input their profile details as a .JSON file and run the script in terminal or shell. Upon command execution, the terminal will display the user's predicted loan approval status with approval probabilities.

The command-line script supports a few different modes:

1. Default mode: The CLI script uses two built-in default sample applicants to predict their loan approval status.
2. Single Applicant mode: The script takes one applicant object as a JSON array and predicts the loan approval status for the applicant. The JSON file path needs to be included in the command.
3. Multiple Applicant mode: The script takes a .csv file with multiple applicants' information as an input and saves the results as `<file>_predictions.csv` in the project root.

Users or applicants can also override the decision threshold and define their own decision threshold if they want. For example: `predict.py --threshold 0.30`



```
Applicant 1: REJECTED P(approved) = 31.5%
Applicant 2: REJECTED P(approved) = 0.0%
(venv) PS D:\Concordia Courses\Summer 2026\COEN 330\COEN 330 Project\COEN330-Machine-Learning-Project> python demo\predict.py
No input given - using built-in sample applicants.

Model : GradientBoosting.pkl
Threshold: 0.4147

Applicant 1: REJECTED P(approved) = 20.4%
Applicant 2: REJECTED P(approved) = 0.0%
(venv) PS D:\Concordia Courses\Summer 2026\COEN 330\COEN 330 Project\COEN330-Machine-Learning-Project> python demo\predict.py --json example.json
Model : GradientBoosting.pkl
Threshold: 0.4147

Applicant 1: REJECTED P(approved) = 31.5%
(venv) PS D:\Concordia Courses\Summer 2026\COEN 330\COEN 330 Project\COEN330-Machine-Learning-Project>
```

Figure 23. CLI loan approval prediction in default and single-applicant modes.

The terminal shows each applicant's predicted status and approval probability, along with the model used and the decision threshold applied.

The above figure 23 demonstrates the implemented CLI's default mode and the single applicant mode using an example.json file located at the project root. We can see that the CLI successfully predicts an applicant's loan approval status with the approval probabilities. The used model and the used decision thresholds are also displayed in the terminal.

Limitations and Future Work

Although the proposed machine learning pipeline achieved strong predictive performance, several limitations should be acknowledged.

First, the dataset used in this study is a publicly available synthetic dataset designed to resemble real-world loan applications. While it is sufficiently large and diverse for evaluating machine learning models, it cannot fully capture the complexity of real lending environments, where approval decisions are influenced by institution-specific policies, changing economic conditions, and customer behaviour. Consequently, the results obtained in this study should be interpreted as an evaluation of the proposed methodology rather than a direct representation of real-world banking systems.

Second, the target variable reflects historical loan approval decisions rather than actual loan repayment outcomes or borrower defaults. As a result, the models learn patterns associated with previous lending decisions instead of directly estimating long-term credit risk. Any historical bias or inconsistency present in the original approval process may therefore be reflected in the trained models.

A notable observation from this project is the strong predictive influence of the **previous_loan_defaults_on_file** feature. Exploratory analysis and permutation importance consistently identified this variable as the most informative predictor across multiple models. Although this feature represents legitimate information rather than data leakage, its dominance may reduce the relative contribution of other applicant characteristics. To better understand this effect, all models were evaluated both with and without this feature. Removing it lowered performance across every model: for gradient boosting, the cross-validated F1 score fell from 0.836 to 0.773. The drop confirms how much the models lean on this single signal, but the remaining F1 of 0.773 also shows that meaningful information stays in the other financial and demographic variables.

Another challenge is the class imbalance within the dataset, where approved loan applications represent a considerably smaller proportion of the observations. Although this issue was addressed through appropriate evaluation metrics, threshold optimization, and class-weighted learning where applicable, predicting the minority class remains more difficult than predicting rejected applications.

Future work could evaluate the proposed pipeline using real loan application data obtained from financial institutions to assess its performance under practical conditions. Incorporating additional financial indicators, such as debt-to-income ratio, repayment history, banking transactions, credit bureau scores, and employment stability, may further improve predictive capability. Future research

could also investigate more advanced ensemble techniques, including XGBoost, LightGBM, and CatBoost, as well as deep learning models for structured financial data. Finally, integrating explainable artificial intelligence techniques together with fairness and bias analysis would improve transparency and support the responsible deployment of machine learning systems in financial decision-making.

Conclusion

This project presented a complete machine learning framework for predicting loan approval decisions using a publicly available dataset of 45,000 loan applications. The problem was formulated as a supervised binary classification task, and an end-to-end workflow was developed, including data preprocessing, exploratory data analysis, feature engineering, model training, hyperparameter tuning, model validation, and performance evaluation. Particular attention was given to preventing data leakage, handling class imbalance, and selecting evaluation metrics that reflect the practical objective of identifying approved loan applications as reliably as possible.

Five classification models, ranging from a simple linear model to more advanced ensemble algorithms, were implemented and compared against a simple dummy baseline using a consistent experimental framework. Hyperparameter tuning was performed through stratified cross-validation, with the F1-score of the approved class used both for model selection and to optimize the final decision threshold. This evaluation strategy provided a more meaningful assessment than relying solely on overall accuracy, given the imbalance present in the dataset.

One of the most significant findings of this study was the dominant influence of the previous loan defaults on file features. Permutation feature importance consistently identified this variable as the strongest predictor across multiple models. To better understand its effect, every model was evaluated both with and without this feature. Although predictive performance declined after its removal, the remaining applicant characteristics continued to provide useful predictive information, demonstrating that loan approval decisions are influenced by multiple financial and demographic factors rather than a single variable.

Overall, the results demonstrate that carefully designed machine learning models can serve as effective decision-support tools for loan approval prediction. Beyond achieving strong predictive performance, the project highlights the importance of rigorous preprocessing, appropriate validation, feature interpretation, and thoughtful evaluation when developing reliable financial prediction systems. While these models are not intended to replace human expertise, they can assist financial institutions by improving consistency, reducing processing time, and supporting more informed lending decisions. Future work involving real-world banking data, additional financial indicators, advanced ensemble techniques, and fairness-aware explainability methods could further enhance the robustness, transparency, and practical applicability of automated loan approval systems.

Team Contribution

Jacques Ivan Cacatian-Martinez

Worked on the coding for the EDA, preprocessing, training, and evaluation. Wrote part of the demo and usage demonstration. Proofread the final report.

Djessica Ndjiongue Djapoum

Wrote part of the Introduction and Motivation, Results and Model Comparison, Error Analysis and Qualitative Discussion and formatted the final report. Proofread the final report.

Mekoupou Eko Candys Eve

Wrote the abstract, part of the Introduction and Motivation, Related Work and Background, Limitations and Future Work, and Conclusion. Proofread the final report.

Aaida Hossain

Wrote the Dataset Description, Preprocessing and Exploratory Data Analysis, Methodology and Models, Validation and Hyperparameter Tuning Strategy, and Experimental Setup. Formatted and proofread the final report.

Dipita Sinha

Worked on the coding for the preprocessing, model training, evaluation, and demo. Wrote part of the demo and usage demonstration section. Proofread the final report.

References

- [1] M. Musharraf, "Loan approval dataset," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/muhammadmusharraf444/loan-approval-dataset> (accessed 14 Jun. 2026).
- [2] T. Wei Lo, "Loan approval classification data," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/taweilo/loan-approval-classification-data/data> (accessed Jun. 15, 2026).
- [3] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Hoboken, NJ, USA: John Wiley & Sons, 2006.
- [4] B. Baesens, T. Van Gestel, S. Viaene, M. Stepanova, J. Suykens, and J. Vanthienen, "Benchmarking state-of-the-art classification algorithms for credit scoring," *Journal of the Operational Research Society*, vol. 54, no. 6, pp. 627–635, Jun. 2003.
- [5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [6] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [8] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, Pittsburgh, PA, USA, 2006, pp. 233–240.
- [9] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, Mar. 2015.
- [10] Statistics Canada, "Life expectancy, at birth and at age 65, by sex, three-year average, Canada, provinces, territories, health regions and peer groups," Table 13-10-0389-01. [Online]. Available: <https://www150.statcan.gc.ca/t1/tbl1/en/tv.action?pid=1310038901> (accessed Jun. 25, 2026).
- [11] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022, pp. 507–520.

- [12] "Permutation feature importance," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/permutation_importance.html (accessed Jun. 25, 2026).
- [13] R. S. Olson, W. La Cava, Z. Mustahsan, A. Varik, and J. H. Moore, "Data-driven advice for applying machine learning to bioinformatics problems," in *Biocomputing 2018*, pp. 192–203, 2018.
- [14] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

GitHub

Link to Repository and Code: [DipiAreum04/Team-1-Project: Applied Machine Learning: Supervised Learning \(Classification Problem\)](#)